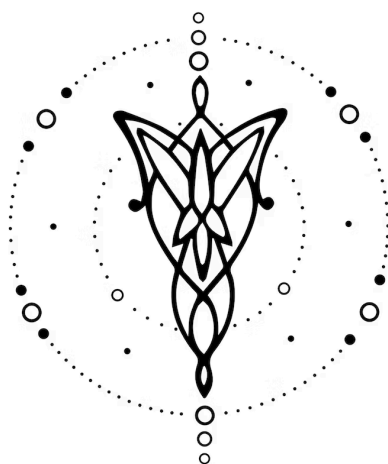




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

V2.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Alexis	Herrera Vegas	64045	aherreravegas@itba.edu.ar
Jeronimo	Esquivel	64756	tesquivel@itba.edu.ar
Andres	Cortese	64612	acortese@itba.edu.ar
Sebastian	Caules	64331	scaules@itba.edu.ar

2. Repositorio

[Link del repositorio](#)

3. Dominio

El objetivo del proyecto es desarrollar un lenguaje de programación específico (DSL) que permita definir, visualizar y manipular agendas de eventos en un calendario, ofreciendo una sintaxis expresiva centrada en el dominio del tiempo.

El lenguaje debe posibilitar la creación de calendarios mensuales o anuales, con soporte para eventos puntuales o recurrentes, y ofrecer herramientas para reemplazar, sobrescribir o extender eventos existentes mediante construcciones de alto nivel.

Cada evento define un instante o período temporal (por ejemplo, una clase semanal o una reunión de media hora), junto con atributos descriptivos como nombre, horario, color o enlace.

El resultado de la interpretación del lenguaje será un calendario HTML visualmente representativo, donde los eventos se renderizan con sus propiedades y colores.

El lenguaje incluye:

- **Declaración de zona horaria**, permitiendo especificar contextos temporales (por ejemplo, timezone America/Argentina/Buenos_Aires).
- **Bloques anidados** para agrupar períodos:

```
timezone GMT-3
define color blue #0000FF
year 2025 {
  month August {
    event Taller on Monday and Friday from 08:00 to 10:00 { color blue }
  }
}
```

- **Eventos expresivos** con palabras clave semánticas: event, on, from, to, every, override, till, since.
- **Sobrescritura de eventos** mediante override para redefinir total o parcialmente eventos previos, conservando su horario y contexto.

El propósito final es crear un lenguaje simple, legible y contextualizado, que acerque la definición de cronogramas a un estilo natural.

4. Construcciones

El lenguaje debe ofrecer las siguientes construcciones y reglas:

I. Definición temporal global

- A. *timezone* <zona>: establece el huso horario, p. ej. *timezone* GMT-3.
- B. *year* <número> { ... }: bloque que agrupa todos los eventos de un año.
- C. *month* <nombre|número> { ... }: bloque anidado dentro de *year*, que define los eventos de un mes específico.

II. Eventos

- A. Definición básica:

`event Título on Monday from 10:00 to 11:00 { color red }`

Permite definir un evento con nombre, día y horario.

III. Sobrescritura y reemplazo

override "Evento": redefine un evento previamente declarado, conservando su horario base:

```
override Taller {
  description "Taller especial"
  color lime
}
```

Reemplaza la instancia del evento en una fecha específica o bajo ciertas condiciones.

IV. Colores y estilos

- A. *declare* <nombre>: <valor> permite registrar colores personalizados (*declare cyan*: #00FFFF).
- B. Se aceptan nombres predefinidos o valores hexadecimales.

V. Comentarios: Inician con // y no afectan la semántica del programa.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). **Evento puntual:**
Programa que declara un evento único en una fecha y horario específicos, verificando que se represente correctamente en el calendario.
- (II). **Evento recurrente semanal:**
Programa que declara un evento que ocurre todos los lunes del mes, comprobando que se repita automáticamente en cada ocurrencia correspondiente.
- (III). **Evento recurrente bisemanal:**
Programa que define un evento recurrente con una periodicidad variable (por ejemplo, cada dos días o semanas), validando que la frecuencia se interprete correctamente.
- (IV). **Concatenación de días:**
Programa que declara un mismo evento en múltiples días de la semana (por ejemplo, lunes y viernes), verificando que se genere una única definición y se duplique visualmente en el calendario.
- (V). **Uso de color declarado:**
Programa que utiliza un color personalizado definido previamente mediante una declaración, comprobando que se aplique correctamente al evento.
- (VI). **Evento recurrente con restricción de ocurrencias:**
Programa que define un evento que ocurre únicamente en ciertas ocurrencias del mes (por ejemplo, la primera o la última semana), asegurando que las reglas de ocurrencia sean respetadas.
- (VII). **Sobrescritura de evento:**
Programa que reemplaza una instancia puntual de un evento recurrente, conservando su horario y demás propiedades originales, pero modificando atributos específicos como título o color.
- (VIII). **Encadenamiento de eventos:**
Programa que establece un evento relativo a otro (por ejemplo, que comience después o finalice antes de otro evento), verificando que la relación temporal entre ambos se mantenga coherente.
- (IX). **Anidamiento temporal:**
Programa que organiza la definición de eventos dentro de bloques jerárquicos de año y mes, validando que el intérprete respete la estructura de contexto temporal.
- (X). **Combinación de construcciones:**
Programa que combina múltiples funcionalidades del lenguaje (recurrentes, sobrescritura, concatenación y colores declarados) en un mismo calendario, comprobando que todas las interacciones sean interpretadas correctamente y generen la salida esperada.

Además, los siguientes casos de prueba de **rechazo**:

- (I). **Estructura sintáctica incorrecta:**
Programa que presenta errores en la estructura del código, como llaves o palabras clave ausentes.
- (II). **Color no declarado o inválido:**
Programa que utiliza un color que no fue declarado previamente o con un formato hexadecimal incorrecto.

- (III). **Fecha inexistente:**
Programa que intenta declarar un evento en una fecha no válida, como el 30 de febrero.
- (IV). **Horario incoherente:**
Programa que indica una hora de finalización anterior a la hora de inicio del evento.
- (V). **Título demasiado extenso:**
Programa que declara un título de evento que supera el límite máximo de 64 caracteres.
- (VI). **Uso de palabra reservada:**
Programa que intenta declarar un color, evento o variable utilizando una palabra reservada del lenguaje.
- (VII). **Sobrescritura inexistente:**
Programa que intenta sobrescribir un evento que no fue declarado previamente.
- (VIII). **Sobrescritura fuera de recurrencia:**
Programa que intenta sobrescribir un evento recurrente en una fecha que no corresponde a su patrón de recurrencia (por ejemplo, un evento recurrente en lunes sobrescrito en un domingo).

6. Ejemplos

Ejemplo 1

Se define el calendario correspondiente al año **2025** para el mes de **octubre**.

Primero se declaran colores personalizados: blue, magenta, cobalt y lime.

A continuación, se definen los siguientes eventos:

- **Taller:** evento recurrente que ocurre los **lunes y viernes** en sus **ocurrencias impares** del mes, de **08:00 a 10:00**, con color **azul**.
- **Seminario Redes:** evento recurrente que ocurre el **primer miércoles** del mes, de **14:00 a 16:00**, con color **magenta**.
- **Consultas:** evento recurrente que ocurre los **martes y jueves** en sus **ocurrencias pares**, de **17:00 a 18:00**, con color **cobalt** (previamente declarado).
- **Taller especial:** sobrescribe (override) la ocurrencia del **17 de octubre** (viernes), reemplazando “Taller” por “Taller especial” y cambiando su color a **lime** (también declarado).

```
timezone gmt-3
define color blue    #0000ff
define color cobalt  #0047ab
define color lime    #32cd32
define color magenta #ff00ff
year 2025 {
    month october {
        event taller on monday and friday from 08:00 to 10:00 { color blue }
        event seminarioredes on wednesday from 14:00 to 16:00 { color magenta }
```

```

    event consultas on tuesday and thursday from 17:00 to 18:00 { color cobalt }

    override taller {
        description "taller especial"
        color lime
    }
}

```

Ejemplo de output en fig. 1

Ejemplo 2

Se crea un calendario correspondiente al año **2025** para el mes de **agosto**.

Dentro de este calendario, se declara un **evento único** llamado “Examen Parcial” el día **10 de agosto**, desde las **09:00 hasta las 11:00 horas**, con color **rojo**.

Posteriormente, se realiza una **sobrescritura** (*override*) sobre esa misma fecha para modificar el título a “Examen Parcial Reprogramado” y el color a **cyan**, manteniendo el horario original.

Este ejemplo permite validar el correcto funcionamiento del mecanismo de sobrescritura, comprobando que se reemplacen únicamente los atributos indicados (título y color) sin alterar las demás propiedades del evento original.

```

timezone gmt-3
define color red   #ff0000
define color cyan  #00ffff

year 2025 {
    month august {
        event examenParcial on 10 from 09:00 to 11:00 { color red }

        override examenParcial {
            description "examen parcial reprogramado"
            color cyan
        }
    }
}

```

Ejemplo 3

Se define un bloque **anual 2026** que contiene dos **bloques mensuales**: **marzo** y **abril**.

- En **marzo**, se declara un evento recurrente “**Consultas**” que ocurre **todos los martes** de **17:00 a 18:00**, utilizando un **color declarado** previamente (cobalt).
- En **abril**, se programa “**Reunión General**” como **evento de período** que abarca **la primera semana del mes** (del día 1 al 7 inclusive), con un color corporativo declarado.

Este ejemplo valida:

1. el **anidamiento temporal** year { month { ... } },
2. la **recurrencia semanal** en un mes específico,
3. la **declaración y reutilización** de colores personalizados, y
4. la representación de un **evento de rango de fechas** (no puntual)

```

timezone gmt-3
define color cobalt    #0047ab
define color corporate #0a84ff

year 2026 {
  month march {
    event consultas on tuesday from 17:00 to 18:00 { color cobalt }
  }

  month april {
    event reuniongeneral on 3 from 09:00 to 12:00 { color corporate }
  }
}

```



FIG. 1: Output del ejemplo 1 en formato html hecho con un programa de prueba en python